BÀI THỰC HÀNH CHƯƠNG 2

I. Tạo cơ sở dữ liệu Book

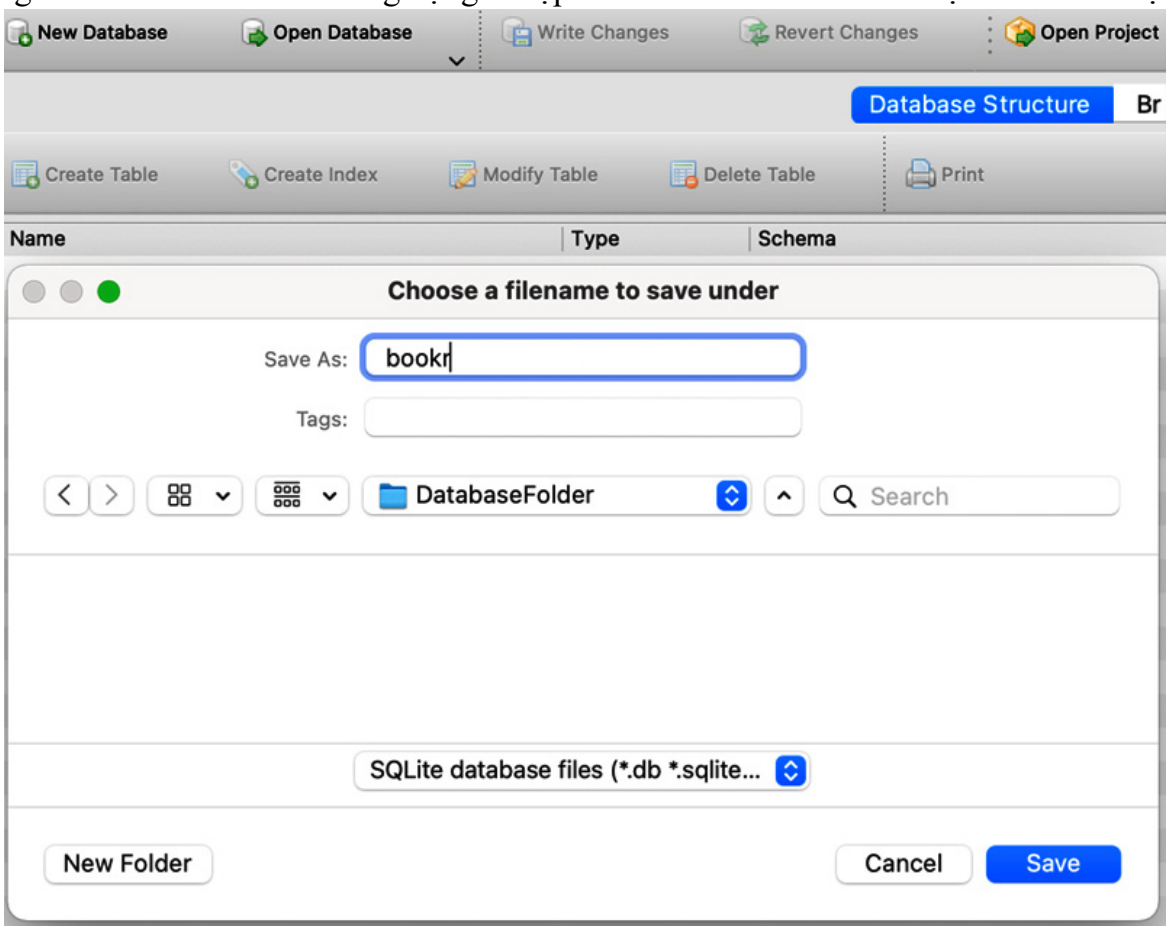
A. Cài đặt DB Browser for SQLite

Trong các bài tập thực hành chúng ta sử dụng cơ sở dữ liệu SQLiet. Để dễ dàng tương tác và quản lý cơ sở dữ liệu SQLite, chúng ta sử dụng phần mềm DB Browser for SQLite. Phần mềm này cho phép quản lý CSDL trực quan và cung cấp giao diện dòng lệnh để thực thi các truy vấn SQL.

Để cài đặt phần mềm DB Browser for SQLite, chúng ta truy cập vào trang https://sqlitebrowser.org, chọn mục Downloads. Sau đó tải về và cài đặt gói tương ứng với hệ điều hành và phần cứng máy tính.

B. Tạo mới cơ sở dữ liệu

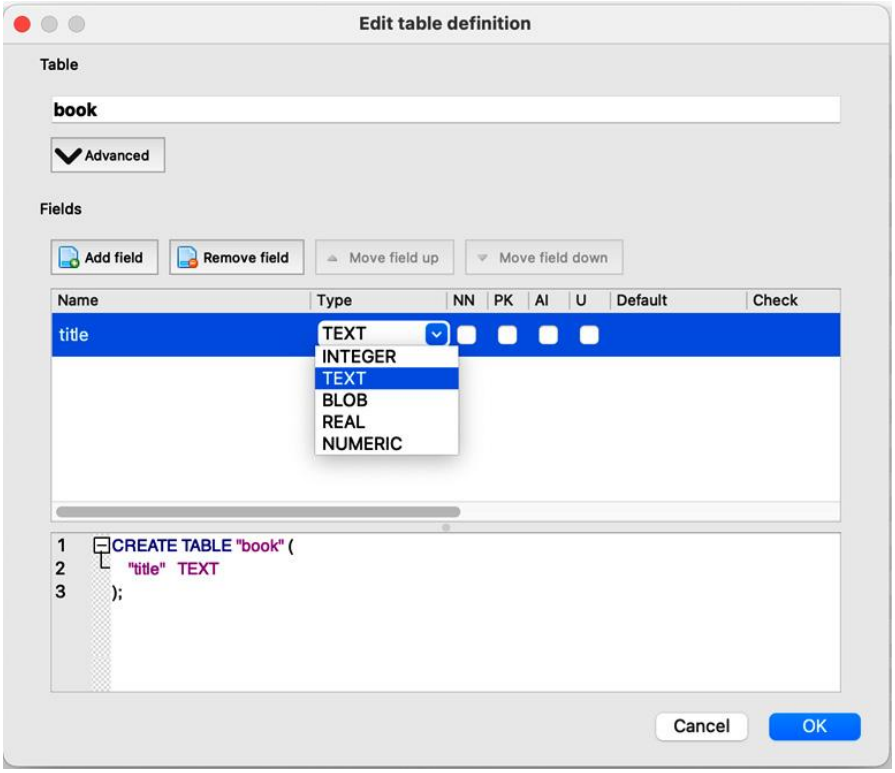
1. Sau khi khởi chạy chương trình **DB Browser (SQLite)**, bấm chuột vào nút **New Database** ở góc trên bên trái cửa sổ ứng dụng. Nhập tên CSDL là **bookr** vào mục **Save As** hoặc .



Hình I-1. Cửa sổ nhập tên và chọn vị trí lưu cơ sở dữ liệu mới của DB Browser for SQLite

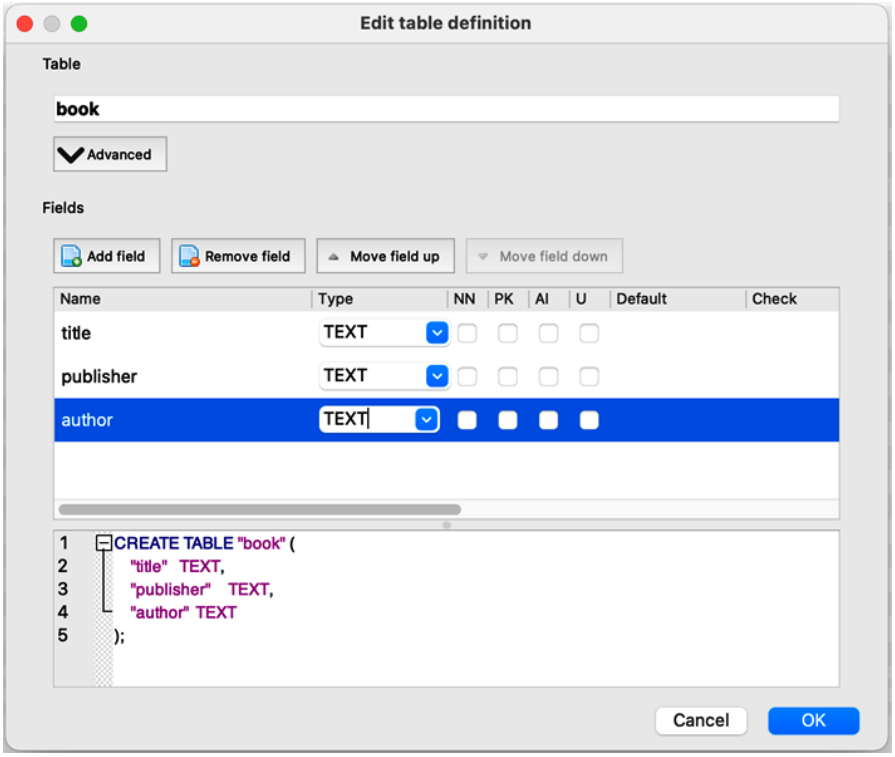
2. Bấm chuột vào nút **Create Table** ở góc trên bên trái cửa sổ và nhập tên bảng mới là **book**.
Lưu ý: *trong trường hợp sau khi tạo mới CSDL ở bước 1 mà ứng dụng tự mở hộp thoại tạo mới bảng thì không cần phải chọn nút Create Table.*

3. Bấm chuột vào nút **Add Field**, nhập tên trường là *title* và chọn kiểu dữ liệu là TEXT ở cột **Type**.



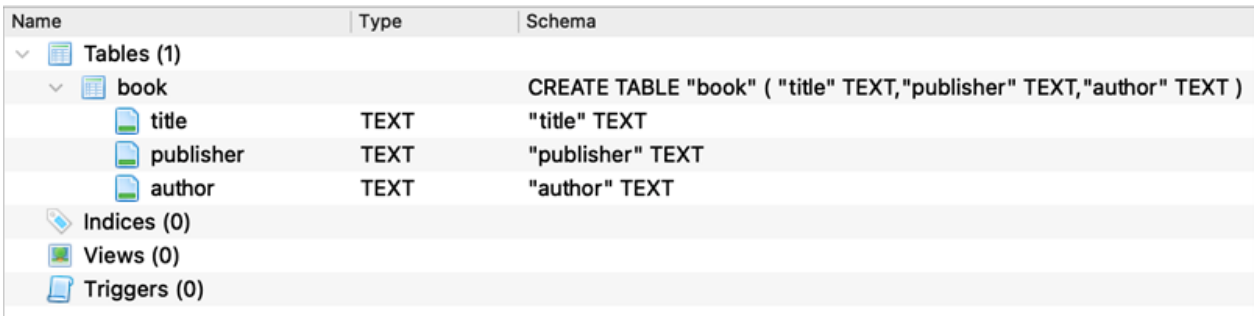
Hình I-2. Thêm trường title với kiểu dữ liệu là TEXT

4. Tương tự, thêm 2 trường nữa vào bảng tên là *publisher và author* có cùng kiểu dữ liệu là TEXT. Sau đó bấm vào nút OK.



Hình I-3. Tạo 2 trường publisher và author cùng có kiểu dữ liệu là TEXT

Kết quả sẽ được một cơ sở dữ liệu **bookr** với một 1 bảng là **book** có 3 trường là *title, publisher và author* như trong Hình I-4.



Hình I-4. Cơ sở dữ liệu bookr với bảng book có 3 trường là title, publisher và author

II. Tạo mới Model và bảng dữ liệu tương ứng

A. Tạo mới Model

Phần này sẽ định nghĩa 4 model gồm:

- **Book**: Chứa thông tin về các cuốn sách.
- **Contributor**: Chứa thông tin về người đóng góp vào việc biên soạn sách, như tác giả, đồng tác giả và biên tập viên.
- **Publisher**: Chứa thông tin về nhà xuất bản.
- **Review**: Chứ thông tin về các bài đánh giá (review) đưa ra bởi người dùng của ứng dụng Bookr.

1. Mở tập tin **models.py** trong thư mục app **reviews** của dự án.
2. Thêm đoạn mã sau vào tập tin **models.py** để tạo model **Publisher** với 3 thuộc tính là *name, website và email*, và một phương thức đặc biệt **__str__()** ghi đè lên phương thức đã định nghĩa sẵn trả về tên của nhà xuất bản.

```
from django.db import models

class Publisher(models.Model):
    """A company that publishes books."""
    name = models.CharField(
        max_length=50,
        help_text="The name of the Publisher.")
    website = models.URLField(
        help_text="The Publisher's website.")
    email = models.EmailField(
        help_text="The Publisher's email address.")

    def __str__(self):
        return self.name
```

3. Tạo thêm model **Contributor** với đoạn mã sau nhập vào tập tin **models.py**. Model này có 3 thuộc tính là *first_names, last_names và email* và một phương thức đặc biệt **__str__()** ghi đè lên phương thức đã định nghĩa sẵn trả về tên (*first_name*) của thành viên đóng góp.

```
class Contributor(models.Model):
    """A contributor to a Book, e.g. author, editor, co-
author."""
    first_names = models.CharField(
        max_length=50,
        help_text="The contributor's first name or names.")
    last_names = models.CharField(
        max_length=50,
        help_text="The contributor's last name or names.")
    email = models.EmailField(help_text="The contact email for
the contributor.")

    def __str__(self):
        return self.first_names
```

4. Tiếp tục thêm đoạn mã sau vào tập tin **models.py** để tạo model **Book**. Model này có 5 thuộc tính là *title, publication_date, isbn, publisher và contributors*. Trong đó, thuộc tính *publisher* liên kết với lớp **Publisher** ở dạng Một – Nhiều và thuộc tính *contributors* liên kết với lớp **Contributor** ở dạng Nhiều – Nhiều thông qua lớp trung gian là **BookContributor**. Lớp này cũng có phương thức đặc biệt **__str__()** ghi đè lên phương thức đã định nghĩa sẵn trả về tiêu đề của cuốn sách. Liên kết **publisher** có tham số `on_delete= models.CASCADE` để chỉ ra rằng nếu xóa nhà xuất bản (publisher) thì các cuốn sách của nhà xuất bản đó cũng sẽ bị xóa cùng.

```
class Book(models.Model):
    """A published book."""
    title = models.CharField(
        max_length=70,
        help_text="The title of the book.")
    publication_date = models.DateField(
        verbose_name="Date the book was published.")
    isbn = models.CharField(
        max_length=20,
        verbose_name="ISBN number of the book.")
    publisher = models.ForeignKey(
        Publisher,
        on_delete=models.CASCADE)
    contributors = models.ManyToManyField(
        'Contributor',
        through="BookContributor")

    def __str__(self):
        return self.title
```

Ta cũng cần bổ sung đoạn mã lệnh để định nghĩa lớp **BookContributor** như sau:

```
class BookContributor(models.Model):
    class ContributionRole(models.TextChoices):
        AUTHOR = "AUTHOR", "Author"
        CO_AUTHOR = "CO_AUTHOR", "Co-Author"
        EDITOR = "EDITOR", "Editor"

    book = models.ForeignKey(
        Book,
        on_delete=models.CASCADE)
    contributor = models.ForeignKey(
        Contributor,
        on_delete=models.CASCADE)
    role = models.CharField(
        verbose_name="The role this contributor had in the
book.",
        choices=ContributionRole.choices, max_length=20)
```

Trong lớp này có thuộc tính *book và contributor* để liên kết với lớp **Book** và **Contributor** tương ứng ở dạng Một – Nhiều. Thuộc tính *role* dùng dể lưu trữ thông tin bổ sung về mối

quan hệ giữa lớp **Book** và **Contributor**. Đây là thuộc tính có giá trị được lựa chọn trong một danh sách sẵn có, danh sách này được định nghĩa trong lớp con **ContributionRole** với 3 giá trị là *AUTHOR, CO_AUTHOR và EDITOR*. Từ khóa **choices** dùng để tham chiếu đến danh sách lựa chọn này. Định nghĩa này rất hữu ích khi tạo Form sử dụng Model này.

5. Cuối cùng thêm đoạn mã sau để định nghĩa model **Review**. Model này có 6 thuộc tính: *content, rating, date_created, date_edited, creator và book*. Trong đó, thuộc tính *date_created* (kiểu thời gian ngày tháng) được tự động gán giá trị là thời điểm tạo ra bản ghi trong cơ sở dữ liệu (tham số `auto_now_add=True`) và thuộc tính *date_edited* có thể trống (tham số `null=True`). Thuộc tính creator liên kết đến model của bảng dữ liệu người dùng của Django ở dạng Một – Nhiều (phương thức **auth.get_user_model()** sẽ trả về model quản lý dữ liệu người dùng này). Thuộc tính *book* liên kết đến model **Book** ở dạng Một – Nhiều. Cả 2 liên kết này đều có tham số `on_delete=models.CASCADE`, có nghĩa là khi xóa người dùng tạo review (creator) hay cuốn sách mà review đề cập đến (book) thì bản ghi review sẽ tự động bị xóa cùng.

```
from django.contrib import auth
class Review(models.Model):
    content = models.TextField(help_text="The Review text.")
    rating = models.IntegerField(help_text="The rating the
reviewer has given.")
    date_created = models.DateTimeField(
        auto_now_add=True,
        help_text="The date and time the review was created.")
    date_edited = models.DateTimeField(
        null=True,
        help_text="The date and time the review was last
edited.")
    creator = models.ForeignKey(
        auth.get_user_model(),
        on_delete=models.CASCADE)
    book = models.ForeignKey(
        Book,
        on_delete=models.CASCADE,
        help_text="The Book that this review is for.")
```

B. Tạo bảng dữ liệu từ Model

Phần này sẽ sử dụng thông tin định nghĩa các model ở trên để tạo bảng dữ liệu thực tế tương ứng sử dụng các lệnh được định nghĩa trong tập tin **manage.py**.

1. Sử dụng lệnh **makemigrations** các mã lệnh để sinh mã tạo các bảng dữ liệu bằng cách chạy dòng lệnh sau trong **terminal**:

```
python manage.py makemigrations reviews
```

Sau khi thực thi lệnh này, kết quả hiển thị ra như sau:

```
reviews/migrations/0002_auto_20191007_0112.py
- Create model Book
- Create model Contributor
```

```
- Create model Review
- Create model BookContributor
- Add field contributors to book
- Add field publisher to book
```

Mã lệnh tạo các bảng dữ liệu được lưu tại thư mục **migrations** của app **Reviews**

2. Tạo tất cả các bảng dữ liệu liên kết với Model (đã sinh mã ở trên) trong cơ sở dữ liệu bằng lệnh **migrate** như sau:

```
python manage.py migrate reviews
```

Kết quả hiển thị sau khi thực thi lệnh này như sau:

```
Operations to perform:
Apply all migrations: reviews
Running migrations:
Applying reviews.0001_initial... OK
```

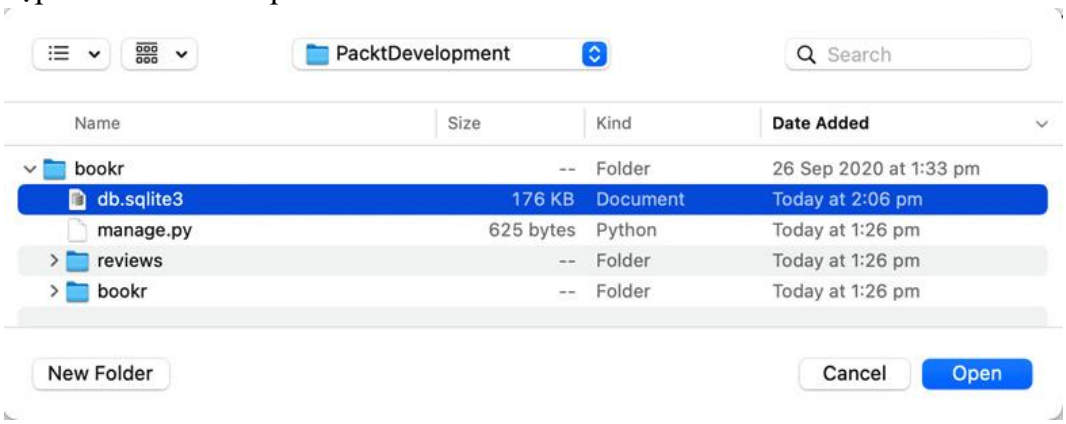
Thông báo này chỉ ra là đã thực hiện tạo các bảng trên cơ sở dữ liệu bảng thành công.

3. Mở ứng dụng **DB Browser for SQLite** và kiểm tra các bảng đã tạo ra trong cơ sở dữ liệu bằng cách bấm chuột vào nút **Open Database** và duyệt đến thư mục dự án **Bookr**.



Hình II-1. Nút Open Database trên thanh công cụ của ứng dụng DB Browser for SQLite

4. Chọn tập tin CSDL db.sqlite3 để mở như Hình II-2.



Hình II-2. Duyệt đến tập tin db.sqlite3 để mở

Sau đó, duyệt đến các bảng đã được tạo trong CSDL này, các bảng vừa được tạo ra sẽ có cấu trúc như trong Hình II-3.

reviews_book		CREATE TABLE "reviews_book" ("id" integer NOT NULL PRIMARY K
id	integer	"id" integer NOT NULL PRIMARY KEY AUTOINCREMENT
title	varchar(70)	"title" varchar(70) NOT NULL
publication_date	date	"publication_date" date NOT NULL
isbn	varchar(20)	"isbn" varchar(20) NOT NULL
publisher_id	integer	"publisher_id" integer NOT NULL
reviews_bookcontributor		CREATE TABLE "reviews_bookcontributor" ("id" integer NOT NULL
id	integer	"id" integer NOT NULL PRIMARY KEY AUTOINCREMENT
role	varchar(20)	"role" varchar(20) NOT NULL
book_id	integer	"book_id" integer NOT NULL
contributor_id	integer	"contributor_id" integer NOT NULL
reviews_contributor		CREATE TABLE "reviews_contributor" ("id" integer NOT NULL PRIM
id	integer	"id" integer NOT NULL PRIMARY KEY AUTOINCREMENT
first_names	varchar(50)	"first_names" varchar(50) NOT NULL
last_names	varchar(50)	"last_names" varchar(50) NOT NULL
email	varchar(254)	"email" varchar(254) NOT NULL
reviews_publisher		CREATE TABLE "reviews_publisher" ("id" integer NOT NULL PRIMA
id	integer	"id" integer NOT NULL PRIMARY KEY AUTOINCREMENT
name	varchar(50)	"name" varchar(50) NOT NULL
website	varchar(200)	"website" varchar(200) NOT NULL
email	varchar(254)	"email" varchar(254) NOT NULL
reviews_review		CREATE TABLE "reviews_review" ("id" integer NOT NULL PRIMARY
id	integer	"id" integer NOT NULL PRIMARY KEY AUTOINCREMENT
content	text	"content" text NOT NULL
rating	integer	"rating" integer NOT NULL
date_created	datetime	"date_created" datetime NOT NULL
date_edited	datetime	"date_edited" datetime
book_id	integer	"book_id" integer NOT NULL
creator_id	integer	"creator_id" integer NOT NULL

Hình II-3. Cấu trúc các bảng của app Reviews đã được tạo trong cơ sở dữ liệu

III. Thao tác với dữ liệu sử dụng Model

Các thao tác ở bài thực hành này này sẽ thực hiện thông qua giao diện dòng lệnh của lệnh **manage.py** shell. Vì thế, trước tiên cần thực thi lệnh sau trong **terminal**.

```
python manage.py shell
```

Sau khi thực thi lệnh trên, terminal sẽ hiển thị nội dung sau:

```
Type "help", "copyright", "credits" or "license" for more
information.
(InteractiveConsole)
>>>
```

Đây là giao diện tương tác, chúng ta có thể nhập nội dung các mã lệnh để tương tác với ứng dụng Bookr sau dấu nhắc >>> .

A. Tạo các bản ghi

Phần này sẽ tạo mới bản ghi trong cơ sở dữ liệu bằng cách lưu một thể hiện (đối tượng) của Model thay vì sử dụng câu truy vấn SQL.

1. Trước tiên, **import** model **Publisher** từ gói review.models

```
>>> from reviews.models import Publisher
```

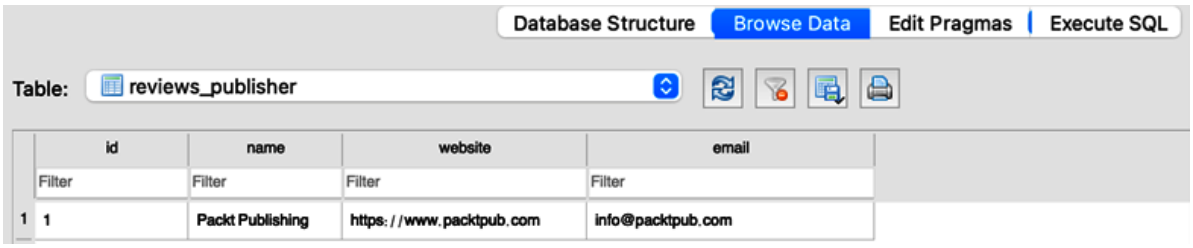
2. Tạo một đối tượng/ thể hiện của lớp **Publisher** bằng cách truyền tất cả giá trị cho các thuộc tính bắt buộc của model **Publisher**, bao gồm: **name**, **website** và **email**.

```
>>> publisher = Publisher(
        name='Packt Publishing',
        website='https://www.packtpub.com',
        email='info@packtpub.com')
```

3. Tiếp theo, để ghi đối tượng này vào CSDL cần gọi phương thức **save**().

```
>>> publisher.save()
```

Sau khi thực thi lệnh này, một bản ghi sẽ được lưu vào trong bảng reviews_publisher như trong Hình III-1.



Hình III-1. Một bản ghi đã được lưu vào trong bảng reviews_publisher

4. Sử dụng đối tượng **publisher** đã tạo ở trên để thực hiện một số điều chỉnh giá trị cho các thuộc tính và lưu lại.

```
>>> publisher.email = 'info@packtpub.com'
>>> publisher.email = 'customersupport@packtpub.com'
>>> publisher.save()
```

Kết quả chỉnh sửa sẽ được lưu lại trong CSDL như Hình III-2.

	id	name	website	email
	Filter	Filter	Filter	Filter
1	1	Packt Publishing	https://www.packtpub.com	customersupport@packtpub.com

Hình III-2. Bản ghi đã được chỉnh sửa theo nội dung mới

B. Sử dụng phương thức create() để tạo một bản ghi

Thay vì phải thực hiện 2 bước là tạo mới và lưu dùng phương thức **save**() để lưu đối tượng vào CSDL như ở phần thước, ta có thể thực hiện chỉ với một bước duy nhất là dùng phương thức **create**(). Sau đây sẽ thử thực hành tạo bản ghi cho model **Contributor**.

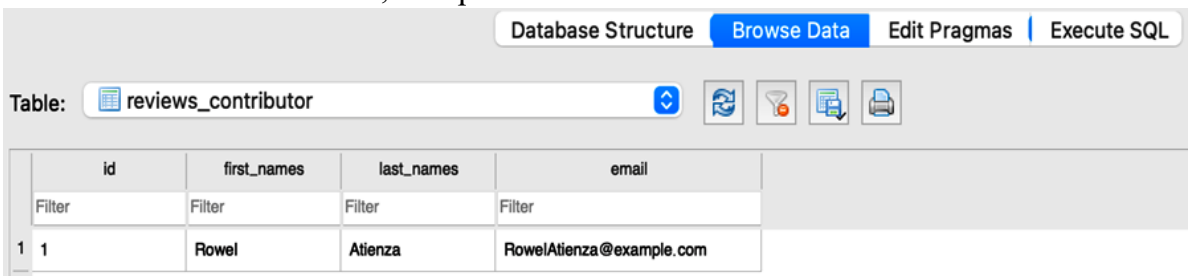
1. Trước tiên, cần **import** lớp **Contributor**:

```
>>> from reviews.models import Contributor
```

2. Gọi phương thức **create**() để tạo đối tượng trong CSDL. Lưu ý cung cấp đầy đủ giá trị cho các thuộc tính bắt buộc (first_names, last_names và email).

```
>>> contributor =
Contributor.objects.create(first_names="Rowel",
last_names="Atienza",
email="RowelAtienza@example.com")
```

3. Mở CSDL trên DB Browser, kết quả sẽ như Hình III-3.



Hình III-3. Bản ghi đã được tạo mới trong bảng reviews_contributor

C. Tạo các bản ghi cho quan hệ Một – Nhiều

Phần này sẽ tạo một bản ghi trong bảng Book, bản ghi này có một khóa ngoại liên kết dạng Một – Nhiều đến bảng Publisher. Vì thế, cần phải truy xuất đối tượng **Publisher** cần liên kết rồi sử dụng đối tượng này làm tham số để tạo bản ghi trong bảng **Book**.

1. Trước tiên, **import** lớp **Publisher**:

```
>>> from reviews.models import Book, Publisher
```

2. Truy xuất đối tượng **Publisher** từ CSDL sử dụng lệnh sau:

```
>>> publisher = Publisher.objects.get(name='Packt Publishing')
```

Trong lệnh này, phương thức **get**() dùng để truy xuất đối tượng từ CSDL.

3. Khi tạo một cuốn sách, cần cung cấp đối tượng kiểu **date** cho thuộc tính **publication_date**. Vì thế cần **import** lớp **date** từ gói **datetime**.

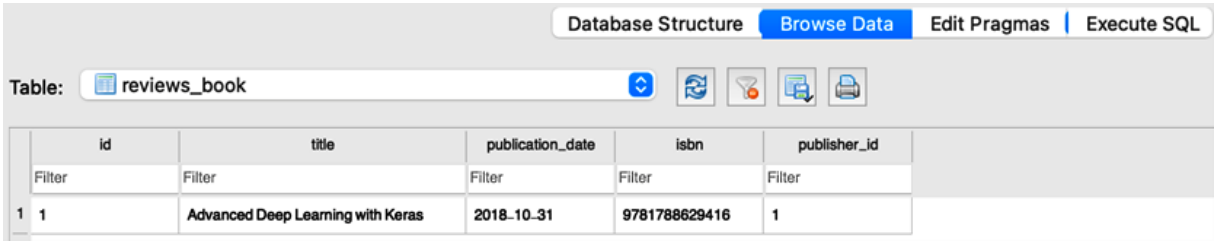
```
>>> from datetime import date
```

4. Sử dụng phương thức **create**() để tạo bản ghi book trong CSDL. Lưu ý truyền tất cả giá trị cho các thuộc tính bắt buộc, gồm **title**, **publication_date, isbn và publisher**.

```
>>> book = Book.objects.create(
```

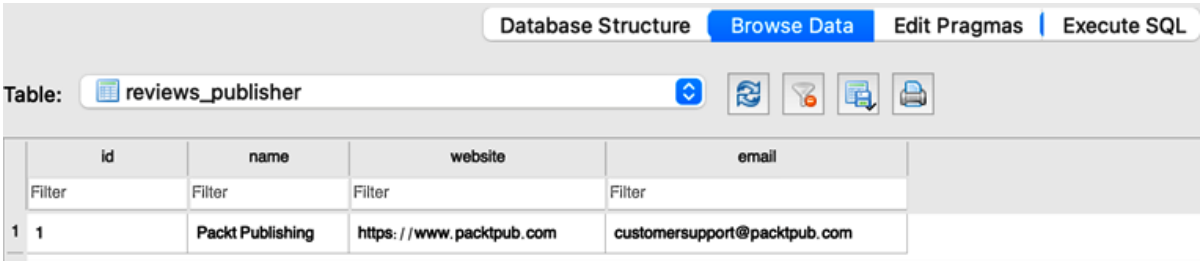
```
        title="Advanced Deep Learning with Keras",
        publication_date=date(2018,10, 31),
        isbn="9781788629416",
        publisher=publisher)
```

Lưu ý: trường **publisher** là khóa ngoại và không được trống (**null**) nên bắt buộc phải truyền đối tượng **publisher**. Nếu không lệnh trên sẽ nhận thông báo lỗi liên quan đến tính toàn vẹn dữ liệu.



Hình III-4. Giá trị khóa ngoại publisher_id trong bảng reviews_book để tham chiếu đến khóa chính

Hình III-4 cho thấy bảng **Book** chứa bản ghi vừa tạo với giá trị khóa ngoại **publisher_id** là 1 dùng để trỏ đến giá trị của khóa chính trong bảng **reviews_publisher** như tại **Hình III-5.**



Hình III-5. Bản ghi có khóa chính là 1 mà khóa ngoại mà bản ghi ở bảng Book trỏ đến

D. Tạo bản ghi với quan hệ Nhiều – Nhiều

Phần này sẽ tạo mới một bảng ghi với quan hệ **Nhiều – Nhiều** giữa bảng **Book** và **Contributor** sử dụng **model BookContributor**.

1. Trường hợp nếu shell đã bị khởi động lại và mất các đối tượng **publisher** và **book** đã tạo ở trên thì cần truy xuất lại các đối tượng này từ **CSDL** sử dụng các lệnh Python sau:

```
>>> from reviews.models import Book
>>> from reviews.models import Contributor
>>> contributor = Contributor.objects.get(first_names='Rowel')
>>> book = Book.objects.get(title="Advanced Deep Learning with
Keras")
```

2. Phương thức để thiết lập quan hệ **Nhiều – Nhiều** giữa các bản ghi là lưu trữ thông tin về quan hệ ở trong model trung gian (hay còn gọi là **model quan hệ**), trong trường hợp này là **model BookContributor**. Giá trị các thuộc tính **book** và **contributor** chính là các đối tượng đã truy xuất ở bước 1.

```
>>> from reviews.models import BookContributor
>>> book_contributor = BookContributor(book=book,
contributor=contributor, role='AUTHOR')
```

```
>>> book_contributor.save()
```

E. Thêm mới một quan hệ Nhiều – Nhiều sử dụng phương thức add()

Thay vì phải tạo ra một đối tượng của model trung gian để thêm mới quan hệ **Nhiều – Nhiều**, phần này sẽ sử dụng phương thức **add**(). Ta có thể sử dụng tham số **through_default** để truyền giá trị kiểu Dictionary với các tham số là giá trị các trường bắt buộc.

Phần này tiếp tục phần trước, thực hiện thêm một **Contributor** cho cuốn sách có tiêu đề *Advanced Deep Learning with Keras*.

1. Nếu shell đã bị khởi động lại thì chạy 2 lệnh sau để nhập và truy xuất dữ liệu cho đối tượng book:

```
>>> from reviews.models import Book, Contributor
>>> book = Book.objects.get(title="Advanced Deep Learning with
Keras")
```

2. Sử dụng phương thức **create**() để tạo một contributor như sau:

```
>>> contributor = Contributor.objects.create(
        first_names='Packt',
        last_names='Example Editor',
        email='PacktEditor@example.com')
```

3. Thêm contributor vừa tạo cho cuốn sách dùng phương thức **add**() với giá trị của thuộc tính role được truyền theo kiểu dữ liệu Dictionary.

```
>>> book.contributors.add(
        contributor,
        through_defaults={'role': 'EDITOR'})
```

Lưu ý: nếu muốn thêm cùng lúc **nhiều contributor** thì có thể sử dụng phương thức **create**() và **set**() với cấu trúc như ví dụ sau:

```
>>> publisher = Publisher.objects.create(name='Pocket Books',
website='https://pocketbookssampleurl.com',email='pocketbook@exa
mple.com')

>>> contributor1 = Contributor.objects.create(
first_names='Stephen', last_names='King',
email='StephenKing@example.com')

>>> contributor2 = Contributor.objects.create(
first_names='Peter',
last_names='Straub', email='PeterStraub@example.com')

>>> book = Book.objects.create(title='The Talisman',
publication_date=date(2012, 9, 25), isbn='9781451697216',
publisher=publisher)

>>> book.contributors.set([contributor1, contributor2],
through_defaults={'role': 'CO_AUTHOR'})
```

F. Đọc dữ liệu từ cơ sở dữ liệu

Phần này sẽ thực hiện đọc dữ liệu từ CSDL sử dụng phương thức **get**().

1. Truy xuất một đối tượng **Publisher** có tên là "*Pocket Books*"

```
>>> from reviews.models import Publisher
>>> publisher = Publisher.objects.get(name='Pocket Books')
```

2. Xem chi tiết đối tượng **Publisher**

```
>>> publisher
<Publisher: Pocket Books>
```

Lưu ý: Nội dung xuất ra ở đây được gọi là chuỗi đại diện của một đối tượng. Đây là kết quả trả về của phương thức **__str__**() của lớp **Publisher** đã được định nghĩa ở phần trước.

3. Có thể truy cập đến từng thuộc tính của đối tượng publisher sử dụng dấu chấm (.) sau tên biến. Ví dụ:

```
>>> publisher.name
'Pocket Books'
>>> publisher.website
'https://pocketbookssampleurl.com'
```

G. Sử dụng phương thức all() để truy xuất tập đối tượng

Phần này thực hành việc sử dụng phương phức **all**() để truy xuất tập các đối tượng

1. Thêm đoạn mã sau để truy xuất tất cả các đối tượng từ bảng **Contributor**

```
>>> from reviews.models import Contributor
>>> Contributor.objects.all()
<QuerySet [<Contributor: Rowel>, <Contributor: Packt>,
<Contributor: Packtp>, <Contributor: Stephen>, <Contributor:
Peter>]>
```

Kết quả thực thi là một tập các đối tượng kiểu **QuerySet**, chứa thông tin các **Contributor**

2. Chúng ta có thể truy cập đến từng đối tượng thông qua chỉ số trong danh sách hoặc lặp để duyệt qua các đối tượng.

```
>>> contributors = Contributor.objects.all()
>>> contributors[0]
<Contributor: Rowel>
>>> contributors[0].first_names
'Rowel'
```

H. Lọc các đối tượng

Phần này thực hiện lấy về một tập các đối tượng thỏa mãn một điều kiện cụ thể

1. Trước tiên, tạo 2 đối tượng **Contributor**

```
>>> from reviews.models import Contributor
>>> Contributor.objects.create(first_names='Peter',
last_names='Wharton', email='PeterWharton@example.com')
>>> Contributor.objects.create(first_names='Peter',
last_names='Tyrrell',  email='PeterTyrrell@example.com')
```

2. Để lọc ra các **Contributor** có tên là ***Peter***, có thể sử dụng phương thức **filter**() như sau:

```
>>> Contributor.objects.filter(first_names='Peter')
<QuerySet [<Contributor: Peter>, <Contributor: Peter>,
<Contributor: Peter>]>
```

3. Để lọc các đối tượng theo các điều kiện so sánh, có thể sử dụng 2 dấu gạch chân (__) và tên phương thức so sánh. Có thể có các phương thức so sánh như:
   - **gt**: Lớn hơn
   - **gte**: Lớn hơn hoặc bằng
   - **lt**: Nhỏ hơn
   - **lte**: Nhỏ hơn hoặc bằng

   Ví dụ như lọc ra các các cuốn sách xuất bản **sau (lớn hơn)** ngày **01/01/2014.**

```
>>> from reviews.models import Book
>>> book = Book.objects.filter(publication_date__gt=date(2014,
1, 1))
>>> book
<QuerySet [<Book: Advanced Deep Learning with Keras>]>
>>> book[0].publication_date
datetime.date(2018, 10, 31)
```

4. Tương tự, có thể thực hiện so khớp văn bản với phương thức **__contains**() (không phân biệt chữ hoa/thường) và **__icontains**() (có phân biệt chữ hoa/thường)
   Ví dụ như lọc ra các cuốn sách mà tiêu đề có chứa chuỗi "*Deep learning*", không phân biệt là chữ hoa hay chữ thường:

```
>>> book = Book.objects.filter(title__contains='Deep learning')
>>> book
<QuerySet [<Book: Advanced Deep Learning with Keras>]>
```

5. Trong trường hợp chúng ta cần lọc các đối tượng theo phương thức loại trừ thì có thể sử dụng phương thức **exclude**().
   Ví dụ muốn lọc ra các **Contributor** có tên không phải là "*Peter*":

```
>>> Contributor.objects.exclude(first_names='Peter')
<QuerySet [<Contributor: Rowel>, <Contributor: Packt>,
<Contributor: Packtp>, <Contributor: Stephen>]>
```

6. Để sắp xếp các đối tượng trong danh sách thì có thể sử dụng phương thức **order_by**().
   Ví dụ sau sắp xếp danh sách các cuốn sách theo ngày xuất bản (**publication_date**):

```
>>> books = Book.objects.order_by("publication_date")
>>> books
<QuerySet [<Book: The Talisman>, <Book: Advanced Deep Learning
with Keras>]>
>>> books[0].publication_date
datetime.date(2012, 9, 25)
>>> books[1].publication_date
datetime.date(2018, 10, 31)
```

   Theo mặc định thì danh sách được sắp xếp theo chiều tăng dần, nếu muốn sắp xếp theo chiều giảm dần thì cần đặt dấu trừ (-) trước tên của trường. Ví dụ:

```
>>> books = Book.objects.order_by("-publication_date")
>>> books
```

```
<QuerySet [<Book: Advanced Deep Learning with Keras>, <Book: The
Talisman>]>
```

7. Thay vì nhận kết quả truy vấn là danh sách các đối tượng, chúng ta có thể chuyển các đối tượng này thành kiểu dữ liệu **Dictionary** bằng cách sử dụng phương thức **values()**:

```
>>> publishers = Publisher.objects.all().values()
>>> publishers
<QuerySet [{'id': 1, 'name': 'Packt Publishing',
'website':'https://www.packtpub.com',
'email':'customersupport@packtpub.com'}, {'id': 2,
'name':'Pocket Books', 'website':
'https://pocketbookssampleurl.com', 'email':
'pocketbook@example.com'}]>
```

I. Truy xuất đối tượng thông qua khóa ngoại

1. Ta có thể truy xuất thông tin thông qua khóa ngoại với chuỗi truy vấn.
   Ví dụ sau lấy ra các cuốn sách có tiêu đề "**The Talisman**" của nhà xuất bản có tên là "**Pocket Books**". Trong đó tên nhà xuất bản có thể tìm thấy trong bảng **Publisher** thông qua khóa ngoại **publisher**.

```
>>> Book.objects.filter(publisher__name='Pocket
Books').filter(title='The Talisman')
<QuerySet [<Book: The Talisman>]>
```

2. Đối với các quan hệ ở dạng **Nhiều – Nhiều**, có thể lấy ra danh sách các đối tượng có mối quan hệ với một đối tượng.
   Ví dụ sau lấy ra danh sách sách các **Contributor** của một cuốn sách có tiêu đề là "**The Talisman"**

```
>>> from reviews.models import Contributor
>>> Contributor.objects.filter(book__title='The Talisman')
<QuerySet [<Contributor: Stephen>, <Contributor: Peter>]>
```

   Nếu xuất phát từ phía đối tượng **Book**, ta có thể lấy bằng phương thức **all**():

```
>>> from reviews.models import Book
>>> book = Book.objects.get(title='The Talisman')
>>> book.contributors.all()
<QuerySet [<Contributor: Stephen>, <Contributor: Peter>]>
```

   Nếu xuất phát từ phía đối tượng **Contributor** (lớp này không có khai báo tường minh về mối quan hệ với lớp **Book**), ta có thể sử dụng phương thức **set**() như sau:

```
>>> from reviews.models import Contributor
>>> contributor = Contributor.objects.get(first_names='Rowel')
>>> contributor.book_set.all()
<QuerySet [<Book: Advanced Deep Learning with Keras>]>
```

J. Điều chỉnh thông tin của bản ghi và xóa bản ghi trong CSDL

1. Ta có thể sử dụng phương thức **update**() để thực hiện điều chỉnh thông tin cho một bản ghi đã có trong CSDL.

Ví dụ sau cập nhật lại tên của các **Contributor** có họ là '*Tyrrell*" thành "*Mike*":

```
>>> from reviews.models import Contributor
```

```
>>> Contributor.objects.filter(last_names='Tyrrell').update(firs
t_names='Mike')
1
```

Số 1 trong kết quả trả về để thông báo rằng có 1 bản ghi đã được điều chỉnh.

2. Tương tự, ta có thể sử dụng phương thức **delete**() để xóa các bản ghi.
Ví dụ sau xóa tất cả các Contributor có họ là "***Wharton***":

```
>>> from reviews.models import Contributor
>>> Contributor.objects.get(last_names='Wharton').delete()
(1, {'reviews.Contributor': 1})
```

K. Thực hiện thêm và cập nhật hàng loạt các đối tượng

1. Thêm hàng loạt đối tượng đồng thời:
Lệnh sau thực hiện thêm đồng thời 3 nhà xuất bản với các thông tin tương ứng:

```
>>> from reviews.models import Publisher
>>> Publisher.objects.bulk_create([
Publisher(name="New Town Publisher",
  website="www.newtownexample.com",
  email='newtow@email.com'),
Publisher(name="Byron Bay Press",
  website="www.byronbayexample.com",
  email='byronbayexample@email.com'),
Publisher(name="Katoomba Publisher",
  website="www.katoombaexample.com",
  email='katoombaexample@email.com')])

[<Publisher: New Town Publisher>, <Publisher: Byron Bay Press>,
<Publisher: Katoomba Publisher>]
```

2. Điều chỉnh thông tin đồng thời cho một loạt các bản ghi bằng phương thức **bulk_update**()
Các lệnh sau thực hiện lấy ra 2 nhà xuất bản có tên là "*'New Town Publisher*" và "*'Byron BayPress*" rồi điều chỉnh thông tin của 2 bản ghi, cuối cùng lưu lại các điều chỉnh này bằng phương thức **bulk_update**():

```
>>> from reviews.models import Publisher
>>> publishers = [Publisher.objects.get(name='New Town
Publisher'), Publisher.objects.get(name='Byron Bay Press')]
>>> publishers[0].website ="www.newsouthwalespublisher.com"
>>> publishers[1].website = "www.newsouthwalespublisher.com"
>>> Publisher.objects.bulk_update(publishers,["website"])
2
```

L. Thực hiện truy vấn phức tạp sử dụng đối tượng Q

Các đối tượng Q được sử dụng để thực hiện các truy vấn phức tạp, đặc biệt là truy vấn có các toán tử AND và OR trong điều kiện WHERE như sau:

```
SELECT * FROM Person WHERE name LIKE "Rob%" OR name LIKE "Bob%";
```

Các đối tượng Q sử dụng kí hiệu **&** và **|** để biểu diễn toán tử **AND** và **OR.**

Khi sử dụng các đối tượng **Q**, truy vấn SELECT ở trên có thể viết thành:

```
Person.objects.get(Q(name__startswith='Rob'), |
Q(name__startswith='Bob')
```

1. Phần này thực hành sử dụng các đối tượng **Q** để thực hiện truy vấn.
   Ví dụ sau sẽ lọc ra tất cả các nhà xuất bản có tên bắt đầu bằng "*New*" hoặc "*Idea*"

```
>>> from django.db.models import Q
>>> from reviews.models import Publisher
>>> Publisher.objects.filter(Q(name__startswith="New")|
Q(name__startswith="Idea"))
<QuerySet [<Publisher: New Town Publisher>]>
```

2. Phần này thực hành sử dụng phương thức **contains()** để kiểm tra xem truy vấn có chứa một đối tượng cụ thể nào không.
   Ví dụ sau sẽ lấy ra danh sách nhà xuất bản từ trong CSDL, sau đó truy xuất nhà xuất bản "*New Town Publisher"* và kiểm tra nhà xuất bản này có trong danh sách xuất ra cử CSDL ban đầu không

```
>>> from reviews.models import Publisher
>>> publishers = Publisher.objects.filter(
Q(name__startswith="New") | Q(name__endswith="Publisher"))
>>> publishers
<QuerySet [<Publisher: New Town Publisher>, <Publisher: Katoomba
Publisher>]>
>>> new_town_publisher = Publisher.objects.get(name='New Town
Publisher')
>>> new_town_publisher
<Publisher: New Town Publisher>
>>> publishers.contains(new_town_publisher)
True
```

IV. Chuẩn bị dữ liệu cho CSDL của dự án Bookr

Phần này hướng dẫn cách để tải về và nhập dữ liệu cho CSDL của dự án **Bookr**.

1. Tạo thư mục **management** và **commands** trong thư mục **bookr/reviews** theo cấu trúc sau:

```
bookr/reviews/management/commands/
```

2. Chép tập tin **loadcsv.py** và **WebDevWithDjangoData.csv** từ địa chỉ sau **`http://packt.live/3pvbCLM`** vào thư mục vừa tạo.
3. Xóa CSDL hiện có để đảm bảo tất cả dữ liệu tạm đã tạo ở các phần trước được xóa hết.

```
rm db.sqlite3
```

4. Tạo lại CSDL sử dụng lệnh **migrate**:

```
python manage.py migrate
```

5. Thực thi tập tin **loadcsv.py** để tạo các bản ghi trong các bảng của CSDL **bookr** bằng lệnh sau:

```
python manage.py loadcsv --
csv reviews/management/commands/WebDevWithDjangoData.csv
```

6. Sử dụng **DB Browser for SQLite** để kiểm tra lại các bảng và dữ liệu của các bảng đã được tạo chưa.

V. Làm việc với View và Template

A. Triển khai một View dựa trên hàm với cấu hình URL tùy biến

Trong tiến tình xử lý một URL của Django, khi một thông điệp HTTP Request gửi đến, Django sẽ tìm đến biến **ROOT_URLCONF** trong tập tin **settings.py**. Biến này thườn có giá trị ở dạng **ROOT_URLCONF = 'tên_dự_án.urls'** hàm ý chỉ đến tập tin **urls.py** nằm trong thư mục của dự án. Theo chỉ dẫn này, Django sẽ đọc trong tập tin **urls.py** ở phần **urlpatterns**. Khi gặp được một mẫu khớp với giá trị URL thì sẽ gọi hàm View tương ứng mẫu đó.

Bài này sẽ thực hành việc cấu hình URL cho View **Welcome**.

1. Đầu tiên, cần kiểm tra biến **ROOT_URLCONF** trong tập tin **bookr/settings.py** trỏ đến đúng tập tin **urls.py** của dự án. Biến **ROOT_URLCONF** được gán như sau:

```
ROOT_URLCONF = 'bookr.urls'
```

2. Mở tập tin **bookr/reviews/views.py** và thêm đoạn mã sau:

```
from django.http import HttpResponse
from .models import Book
def welcome_view(request):
    message = f"<html><h1>Welcome to Bookr!</h1>
    <p>{Book.objects.count()} books and counting!</p></html>"
    return HttpResponse(message)
```

Như chúng ta đã biết ở Chương 1, trong đoạn mã trên, tham số request chứa đối tượng HTTPRequest cho phép cung cấp thông tin về nội dung thông điệp HTTP request gửi đến từ trình khách ứng dụng web.

3. Tạo một ánh xạ URL để gọi đến hàm **welcome_view()** mới định nghĩa. Mở tập tin **bookr/urls.py** và điều chỉnh lại nội dung như sau:

```
from django.contrib import admin
from django.urls import include, path
import reviews.views
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('reviews.urls'))
]
```

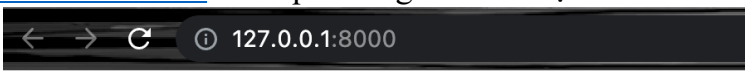
Các ánh xạ URL khai báo trong biến urlpatterns gồm có:

- **path('admin/', admin.site.urls):** định tuyến đến urls phục vụ app admin khi đường dẫn URL đến mục admin (ở dạng ***http://0.0.0.0:8000/admin***)
- **path('', include('reviews.urls')):** trong đó tham số đầu tiên là chuỗi rỗng, nghĩa là nếu đường dẫn URL không chỉ đến mục con nào thì sẽ tra cứu URL trong tập tin **bookr/reviews/urls.py** (qua khai báo **include('reviews.urls')**). Hàm **include()** ở đây dùng để thêm các khai báo ánh xạ URL từ tập tin bên ngoài tập tin **urls.py** chung của cả hệ thống. Đây là cách phổ biến để giao việc thiết lập các ánh xạ URL về cho từng App trong hệ thống.

4. Tạo mới tập tin **urls.py** trong thư mục **bookr/reviews/** và thêm các dòng mã khai báo ánh xạ URL như sau:

```
from django.contrib import admin
from django.urls import path
from . import views
urlpatterns = [
    path('', views.welcome_view, name='welcome_view'),
]
```

5. Sau khi lưu các tập tin trên, khởi chạy trình chủ phát triển ứng dụng Django và truy cập đến trang http://127.0.0.1:8000. Kết quả trang web sẽ hiện ra như sau:



Welcome to Bookr!

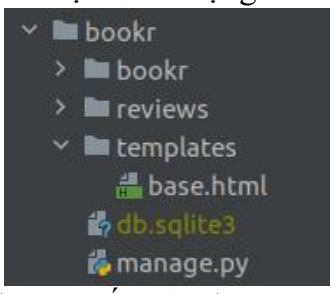
18 books and counting!

Hình V-1. Trang Welcome to Bookr! với số cuốn sách có trong CSDL

B. Sử dụng Template để hiển thị thông điệp chào mừng

Phần này sẽ tạo một template Django và hiển thị thông điệp chào mừng sử dụng Template.

1. Tạo một thư mục là **templates** trong thư mục dự án **Bookr** và trong thư mục này tạo một tập tin là **base.html.** Cấu trúc thư mục sẽ có dạng như hình sau:



*Hình V-2. Cấu trúc thư mục **bookr***

2. Thêm dòng mã được tô đậm trong đoạn mã sau (thuộc phần biến **TEMPLATES** trong tập tin **settings.py**).

```
import os
TEMPLATES = [
{
    'BACKEND':'django.template.backends.django.DjangoTemplates',
    'DIRS': [os.path.join(BASE_DIR, 'templates')],
    'APP_DIRS': True,
    'OPTIONS': {
        'context_processors':[
            'django.template.context_processors.debug',
            'django.template.context_processors.request',
            'django.contrib.auth.context_processors.auth',
```

```
'django.contrib.messages.context_processors.messages',
            ],},
},]
```

Lưu ý: *Khi giá trị của tùy chọn* ***'DIRS'*** *để trống thì Django sẽ tự động tìm kiếm template ở các thư mục* ***templates*** *của các App, nhưng khi tùy chọn này được gán giá trị cụ thể như ở trên thì Django chỉ tìm ở trong thư mục có địa chỉ được gán này thôi. Do vậy, ở bài thực hành Chương 1, chúng ta đã sử dụng Template đặt trong thư mục* ***templates*** *của App Reviews mà không cần thiết đặt giá trị cho tùy chọn* ***'DIRS'*** *này. Nhưng khi tùy chọn này được gán thì Django sẽ* ***không*** *tìm kiếm template trong thư mục* ***reviews/templates*** *nữa.*

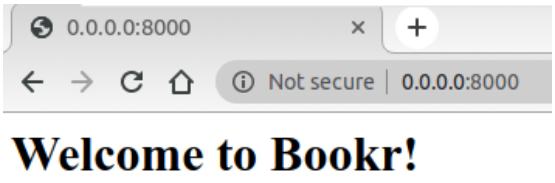
3. Thêm đoạn mã sau vào tập tin **base.html** và lưu vào trong thư mục **bookr/templates**:

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>Home Page</title>
</head>
<body>
    <h1>Welcome to Bookr!</h1>
</body>
</html>
```

4. Thay đổi nội dung tập tin **bookr/reviews/views.py** thành như sau:

```
from django.shortcuts import render
def welcome_view (request):
return render(request, 'base.html')
```

5. Lưu các tập tin và khởi chạy trình chủ web. Kết quả khi truy cập đến địa chỉ http://127.0.0.1:8000/ sẽ như sau:



Hình V-3. Trang chủ Welcome to Bookr!

C. Hiển thị danh sách sách và reviews

Phần này sẽ tạo một trang web hiển thị danh sách các cuốn sách và các đánh giá.

1. Tạo một tập tin tên là **utils.py** trong thư mục **books/reviews/** và thêm đoạn mã sau:

```
def average_rating(rating_list):
    if not rating_list:
        return 0
    return round(sum(rating_list) / len(rating_list))
```

Phương thức này sẽ được dùng để tính giá trị rating trung bình của một cuốn sách.

2. Xóa toàn bộ mã nguồn trong tập tin bookr/reviews/views.py và thay bằng đoạn mã sau:

```
from django.shortcuts import render, get_object_or_404
```

```
from .models import Book
from .utils import average_rating
def index(request):
    return render(request, "base.html")
def book_list(request):
    books = Book.objects.all()
    book_list = []
    for book in books:
        reviews = book.review_set.all()
        if reviews:
            book_rating = average_rating([review.rating for
review in reviews])
            number_of_reviews = len(reviews)
        else:
            book_rating = None
            number_of_reviews = 0
        book_list.append({'book': book,
                          'book_rating': book_rating,
                          'number_of_reviews':
number_of_reviews})

    context = {
        'book_list': book_list
    }
    return render(request, 'reviews/book_list.html', context)
```

Hàm **book_list()** sẽ truy xuất danh sách tất cả các cuốn sách từ CSDL. Sau đó sẽ duyệt qua từng cuốn sách và tính giá trình rating trung bình cho mỗi cuốn. Tất cả các thông tin này được thêm vào trong danh sách tên là **book_list** và biến **book_list** này được dùng để tạo biến **context**. Cuối cùng, biến **context** này được truyền vào hàm **render().**

3. Tạo tập tin template **book_list.html** trong thư mục **bookr/reviews/templates/reviews/** và nhập vào đoạn mã sau:

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>Bookr</title>
</head>
    <body>
        <h1>Book Review application</h1>
        <hr>
        <ul>
          {% for item in book_list %}
          <li>
              <span>Title: </span>
              <span>{{ item.book.title }}</span>
              <br>
```

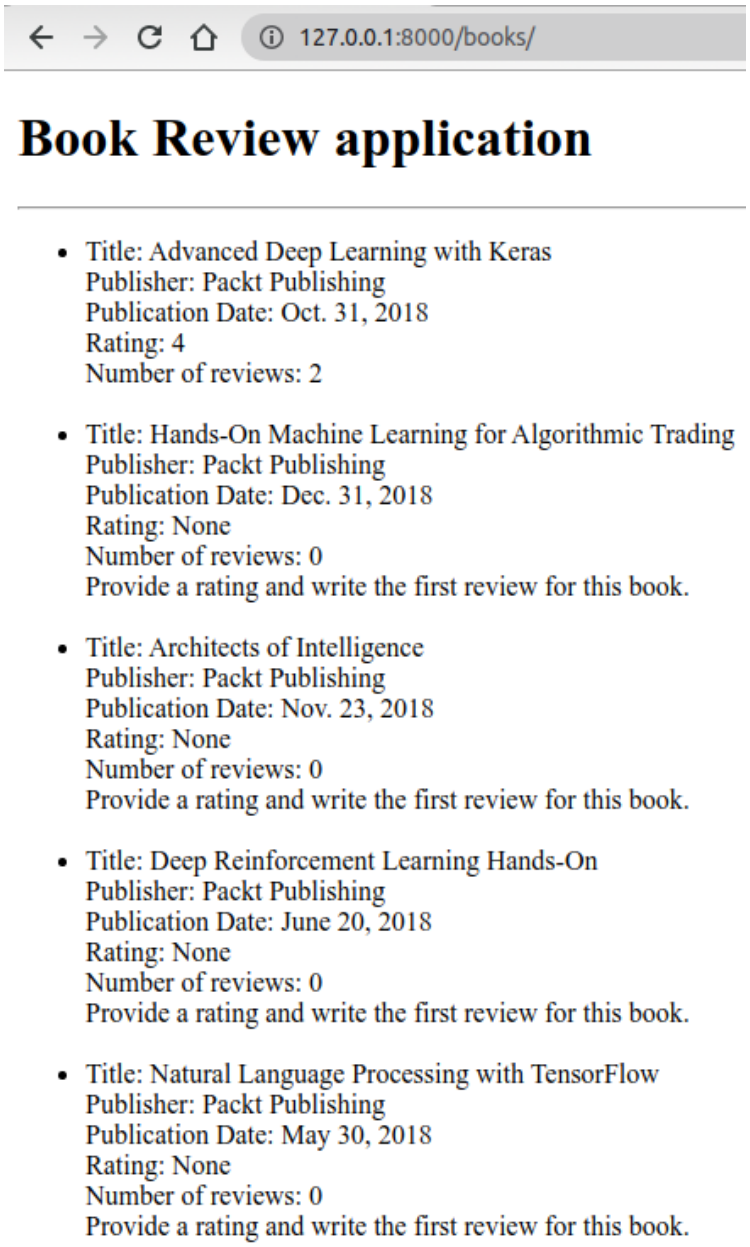
```
                <span>Publisher: </span>
                <span>{{ item.book.publisher }}</span>
                <br>
                <span>Publication Date: </span>
                <span>{{ item.book.publication_date }}</span>
                <br>
                <span>Rating: </span>
                <span>{{ item.book_rating }}</span>
                <br>
                <span>Number of reviews: </span>
                <span>{{ item.number_of_reviews }}</span>
                <br>
                {% if item.book_rating is null %}
                <span>Provide a rating and write the first review
for this book.</span>
                <br>
                {% endif %}
            </li>
          <br>
            {% endfor %}
          </ul>
    </body>
</html>
```

Đoạn mã này có chứa các tag để thực hiện duyệt qua tất cả các cuốn sách trong biến **book_list** và hiển thị các trường thông tin của cuốn sách theo định dạng HTML.

4. Thêm đoạn mã ánh xạ URL sau vào trong tập tin **bookr/reviews/urls.py** để ánh xạ đến hàm **book_list.**

```
from django.urls import path
from . import views
urlpatterns = [
    path('books/', views.book_list, name='book_list'),
]
```

5. Lưu các tập tin lại, chạy trình chủ web và truy cập đến trang http://0.0.0.0:8000/books/. Kết quả sẽ như sau:



Hình V-4. Danh sách các cuốn sách

D. Sử dụng tính kế thừa của Template

Phần này sẽ sử dụng tính kế thừa của Template để thừa kế lại từ trang template cơ sở và sử dụng trong Template **book_list** để liệt kê các cuốn sách. Chúng ta cũng sử dụng các thành phần của thư viện Bootstrap để thêm một thanh điều hướng ở trên đầu trang web.

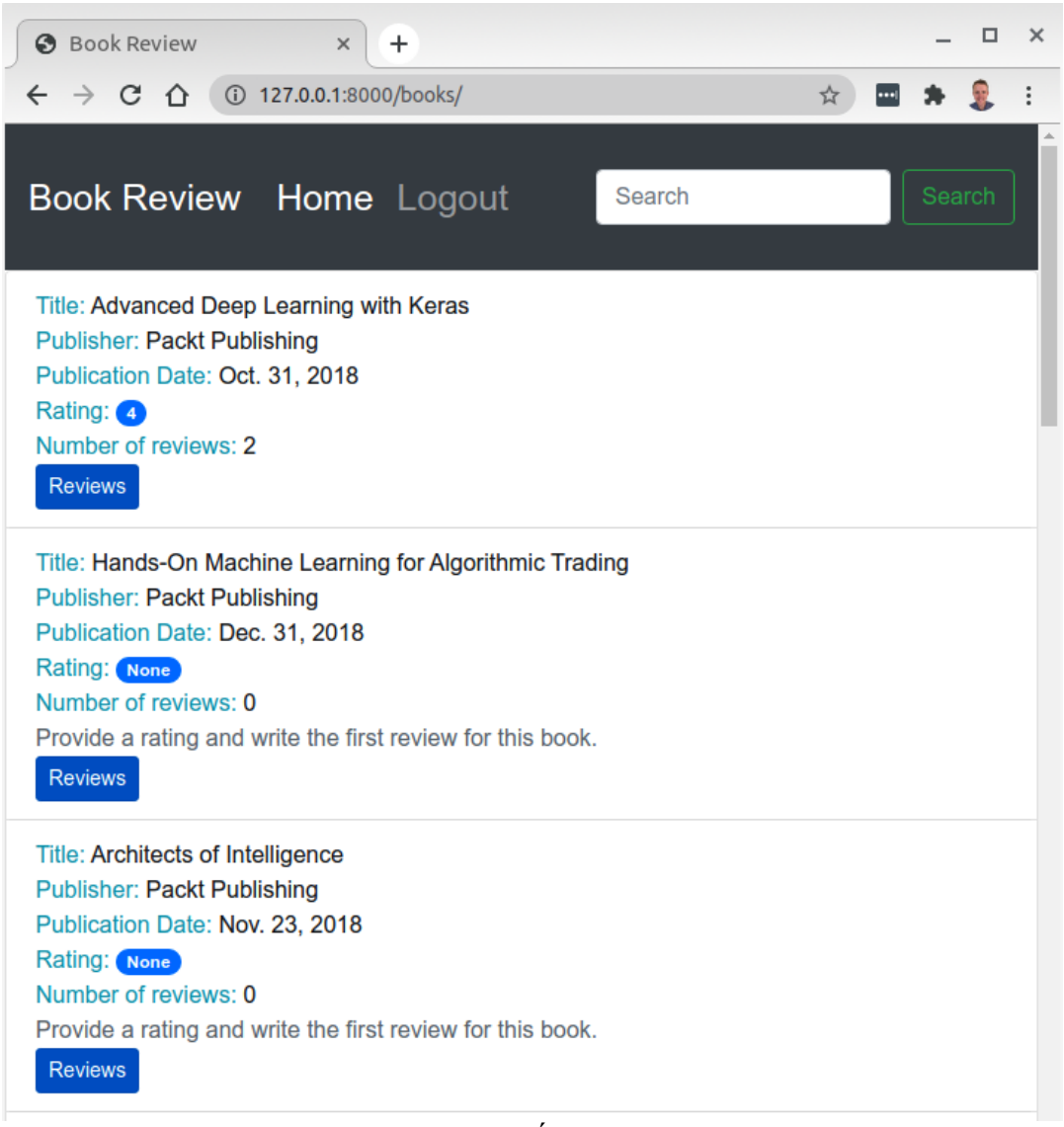
1. Mở tập tin **base.html** trong thư mục **bookr/templates/.** Xóa nội dung hiện có của tập tin và thay bằng đoạn mã có trong tập tin https://github.com/PacktPublishing/Web-Development-with-Django-Second-Edition/blob/main/Chapter03/Exercise3.04/bookr/reviews/templates/reviews/base.html
2. Mở tập tin template **bookr/reviews/templates/reviews/books_list.html**. Xóa nội dung hiện có của tập tin và thay bằng đoạn mã sau:

```
{% extends 'base.html' %}
```

```
{% block content %}
<ul class="list-group">
  {% for item in book_list %}
  <li class="list-group-item">
      <span class="text-info">Title: </span> <span>{{
item.book.title }}</span>
      <br>
      <span class="text-info">Publisher: </span>
      <span>{{ item.book.publisher }}</span>
      <br>
      <span class="text-info">Publication Date: </span>
      <span>{{ item.book.publication_date }}</span>
      <br>
      <span class="text-info">Rating: </span>
      <span class="badge badge-primary badge-pill">
{{ item.book_rating }}</span>
      <br>
      <span class="text-info">Number of reviews: </span>
      <span>{{ item.number_of_reviews }}</span>
      <br>
      {% if item.book_rating is null %}
      <span class="text-secondary">Provide a rating and write
the first review for this book.</span>
      <br>
      {% endif %}
      <a class="btn btn-primary btn-sm active" role="button"
aria-pressed="true" href="/book/{{ item.book.id }}/">
Reviews</a></li>
  </li>
  {% endfor %}
</ul>
{% endblock %}
```

Template này được khai báo kế thừa từ tập tin **base.html** và có thêm một số thành phần định dạng khác để liệt kê danh sách các cuốn sách.

3. Sau khi thêm 2 Template mới, mở trang http://127.0.0.1:8000/books/ và kết quả sẽ là trang web đã được định dạng lại như sau:



Hình V-5. Trang danh sách các cuốn sách sau khi được định dạng lại